

Programación concurrente y semáforos en Java

- En Java es posible ejecutar tareas en paralelo, utilizando hebras de control (hilos, threads). Este modo de programación permite tener un espacio de memoria, código o recursos compartidos. Tiene la ventaja que su ejecución resulta más económica que un proceso completo.

Semáforos

- Java cuenta con semáforos implícitos de la forma:

```
Object mutex = new Object();  
/* ... */  
Synchronized (mutex){  
/* ... */  
}
```

Que solo pueden ser utilizados para exclusión mutua.

Solo una hebra de control puede ejecutarse en el bloque synchronized en un momento dado.

Ejemplo del uso de la palabra Synchronized

```
import java.io.*;
class Banco {
    public static void main ( String args[]) {
        try {
            // Declaramos los dos montones de billetes
            Contador co1 = new Contador ();
            Contador co2 = new Contador ();
            // Declaramos los dos cajeros
            Cajero c1 = new Cajero(co1);
            Cajero c2 = new Cajero(co2);
            // Se ponen a contar..
            c1.start();
            c2.start();
            c1.join();
            c2.join();
        } catch ( Exception e ){
            e.printStackTrace(); }
    }
}
```

Clase Contador: Cuenta billetes y almacena la suma

```
class Contador {  
    int numBilletes = 0 ;  
    long suma = 0 ;  
    final int TOTAL_BILLETES = 10000 ;  
    final int VALOR_BILLETES = 200 ;  
    void cuenta () {  
        // A contar la suma de los billetes  
        for ( numBilletes =0 ; numBilletes < TOTAL_BILLETES;  
            numBilletes ++ ) {  
            suma += VALOR_BILLETES ;  
            // Billetes de 200 pesos Thread.yield(); }  
            System.out.println ( numBilletes+ " suman : "+ suma +  
                " pesos");  
        }  
    }  
}
```

Clase Cajero: Recibe cierta cantidad de billetes para contar

```
class Cajero extends Thread {
```

```
    Contador contadorCajero ;
```

```
    Cajero ( Contador paramContador )
```

```
    {
```

```
        contadorCajero = paramContador ;
```

```
    }
```

```
    public void run ()
```

```
    {
```

```
        contadorCajero.cuenta();
```

```
    }
```

```
}
```

Resultado:

- 10000 suman 2000000 pesos
- 10000 suman 2000000 pesos

Es correcto, dado que cada cajero tiene su cantidad de billetes para contar.

Compartiendo el recurso

- Ahora supongamos que los dos cajeros deben contar del mismo montón, o sea, lo comparten y por tanto, la suma de lo que haya contado cada uno debe ser el resultado total.
- Para ello, modificaremos el código añadiendo lo siguiente

Declaramos los dos montones de billetes :

```
▫ Contador co1 = new Contador ();  
  // Ahora sobra, Contador co2 = new Contador ();  
  // Declaramos los dos cajeros y el mismo montón.  
  Cajero c1 = new Cajero(co1);  
  Cajero c2 = new Cajero(co1);
```

Con este cambio obtenemos:

- 10000 suman: 2000200 pesos
- 10001 suman: 2000200 pesos

- El resultado anterior es incorrecto
- Por tanto, debemos utilizar un mecanismo de sincronización que garantice que cuando un cajero cuente un billete y lo sume, el otro no pueda intentar coger el mismo billete y sumarlo. La solución que ofrece Java para resolver este problema es de lo más simple y eficiente, utilizando la cláusula **synchronized** en la declaración del método donde se realiza la tarea "crítica".
- Por tanto, cambiaremos el método void cuenta()
por:
synchronized void cuenta ()

- Si realizamos ese cambio, obtenemos el siguiente resultado:
- 10000 suman : 2000000 pesos
- 10000 suman : 4000000 pesos
- Esto ocurre porque no se inicializa la variable suma antes del ciclo que cuenta los billetes, por lo que el segundo cajero continúa la suma en donde la dejó el anterior.

Inicializando el contador dentro de cada

- Si modificamos el código e incluimos la inicialización, tendremos:

```
void cuenta () {  
    // Cada cajero cuenta lo suyo ...  
    suma = 0 ;  
    // A contar la suma de los billetes  
    for ( numBilletes =0 ; numBilletes < TOTAL_BILLETES ;numBilletes ++ )  
    {  
        suma += VALOR_BILLETES ;  
        Thread.yield();  
    }  
}
```

A partir de este momento obtenemos el siguiente resultado esperado tal y como detallamos:

10000 suman : 2000000 pesos

10000 suman : 2000000 pesos

- Otra forma de realizar la sincronización consiste en declarar el objeto compartido como sincronizado en vez del método que lo contiene. Se realiza entonces el siguiente cambio:

```
public void run(){  
    contadorCajero.cuenta();  
}
```

por:

```
public void run(){  
    synchronized (contadorCajero ) {  
        contadorCajero.cuenta();  
    }  
}
```

Regiones Críticas

- Aún cuando los semáforos representan una solución sencilla para la sincronización de procesos concurrentes, no son una solución definitiva
- Debido al anidamiento de secciones críticas tienden a dificultar su uso y a generar complejos protocolos. La solución propuesta es el uso de regiones críticas.
- Desarrolladas por C.A.R. Hoare y P. Brinch Hansen.

Regiones críticas

- Las regiones críticas son una extensión al uso de semáforos.
- Ahora la región crítica representa el código donde se accede al recurso compartido, y se colocan en una zona fuera del programa.
- La notación propuesta por Hoare es:
 - Se compone por una variable compartida o recurso v y la región crítica C
 - With v do C

Monitores

- El siguiente nivel dentro de la solución a los problemas de exclusión mutua y sincronización entre procesos concurrentes fue desarrollado por C.A.R. Hoare y P. Brinch Hansen.
- Ahora se considera como recurso compartido no únicamente las variables compartidas, sino también a los procedimientos y funciones que actúan sobre las variables

Seudocódigo

- Notación propuesta por Hoare (Simula 67)

monitorname: monitor

begin //declaraciones de datos locales del monitor

 procedure //procname { parametros formales }

 begin // cuerpo del procedimiento

 ... // otros procedimiento locales del monitor

 end

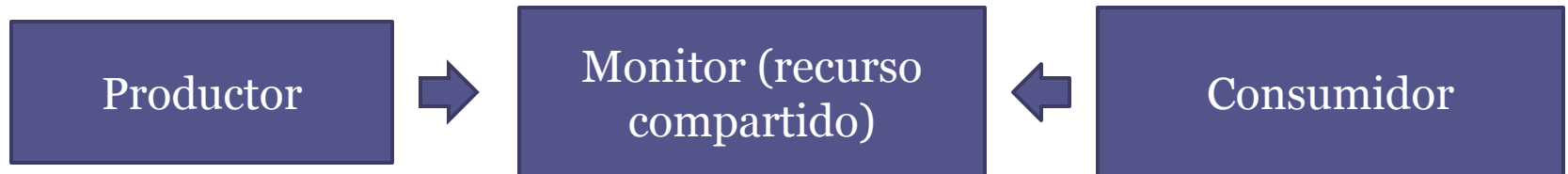
 ...

 ..

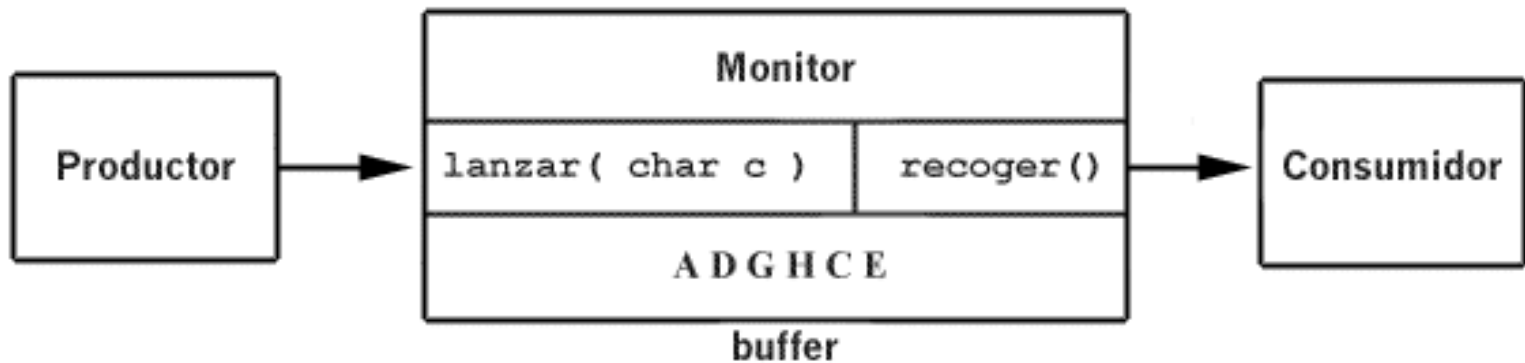
//inicialización de los datos locales del monitor

end

Monitores



Ejemplo



Lectura y Escritura de Archivos

```
import java.io.BufferedReader;
import java.io.BufferedOutputStream;
import java.io.FileInputStream;
import java.io.FileOutputStream;
```

```
public class LecturaEscrituraArchivos {
```

```
    public static void main(String args[]){
        copiaArchivo("c:/archivoEntrada.txt", "c:/archivoSalida.txt");
    }
```

•

```
public static void copiaArchivo (String archivoLectura, String archivoEscritura){
    try{
        FileInputStream fileInput = new FileInputStream(archivoLectura);
        BufferedInputStream bufferedInput = new BufferedInputStream(fileInput);

        FileOutputStream fileOutput = new FileOutputStream(archivoEscritura);
        BufferedOutputStream bufferedOutput = new
BufferedOutputStream(fileOutput);
        byte [] array = new byte [1];
        int leidos= bufferedInput.read(array);
        while(leidos > 0){
            bufferedOutput.write(array);
            leidos=bufferedInput.read(array);
        }
        bufferedInput.close();
        bufferedOutput.close();

    }catch(Exception e){
        e.printStackTrace();
    }
}
```

RandomAccessFile

- Mediante los objetos de esta clase utilizamos archivos binarios mediante un acceso aleatorio, tanto para lectura como para escritura.
- Existe un índice que nos indica en qué posición del archivo nos encontramos, y con el que se puede trabajar para posicionarse en el archivo.
- `RandomAccessFile(String nombre, String modo)`
 - `nombre`: cadena identificadora del archivo
 - `modo`: si será de lectura y/o escritura

Ejemplo de algunos métodos de escritura

- La escritura del archivo se realiza con una función que depende el tipo de datos que se desee escribir.
- *void write(byte b[], int ini, int len);* Escribe *len* caracteres del vector *b*.
- *void write(int i);* Escribe la parte baja de *i* (un byte) en el flujo.
- *void writeBoolean(boolean b);* Escribe el boolean *b* como un byte.
- *void writeByte(int i);* Escribe *i* como un byte.
- *void writeBytes(String s);* Escribe la cadena *s* tratada como bytes, no caracteres.
- *void writeChar(int i);* Escribe *i* como 1 byte.
- *void writeChars(String s);* Escribe la cadena *s*.
- *void writeDouble(double d);* Convierte *d* a *long* y le escribe como 8 bytes.
- *void writeFloat(float f);* Convierte *f* a entero y le escribe como 4 bytes.
- *void writeInt(int i);* Escribe *i* como 4 bytes.
- *void writeLong(long v);* Escribe *v* como 8 bytes.
- *void writeShort(int i);* Escribe *i* como 2 bytes.
- *void writeUTF(String s);* Escribe la cadenas UTF
- Para la lectura existen métodos análogos para leer cada uno de los tipos de datos.